

Build em C++: Makefile e CMake

Tutorial prático para o Trabalho Prático 1

1. Por que um sistema de build?

Quando seu projeto tem um único arquivo, compilar na mão funciona bem:

```
g++ -std=c++17 -Wall -o meu_programa main.cpp
```

Com quatro, cinco ou mais arquivos, isso vira rapidamente:

```
g++ -std=c++17 -Wall -o sistema main.cpp livro.cpp usuario.cpp emprestimo.cpp pedido.cpp ...
```

Um **sistema de build** automatiza esse comando: você digita `make` ou `cmake --build build` e ele descobre sozinho quais arquivos compilar, com quais flags, e na ordem certa. Além disso, o CMake gera informações que o VS Code usa para IntelliSense e navegação de código.

Neste tutorial você aprenderá o básico de cada ferramenta e como usar LLM para gerar os arquivos do **seu** projeto.

2. Makefile

2.1 Como funciona

Um Makefile é um arquivo de texto chamado exatamente **Makefile** (sem extensão) na raiz do projeto. Ele descreve **regras**: o que gerar, de onde, e como.

A estrutura de uma regra é:

```
alvo: dependências
<TAB> comando para gerar o alvo

# ATENÇÃO: a indentação DEVE ser uma TAB real, não espaços.
```

Quando você digita `make`, ele lê o Makefile, verifica quais alvos estão desatualizados em relação às suas dependências, e executa apenas os comandos necessários. O alvo especial `all` é executado por padrão.

2.2 Exemplo mínimo — compilação simples

Para o TP1, a versão mais simples compila todos os arquivos `.cpp` em um único comando. É suficiente para este trabalho:

```
# Makefile — versão simples (todos os .cpp de uma vez)
# Adequada para o TP1; a partir do TP2 usaremos compilação incremental.

CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -g
TARGET   = meu_sistema
SRCS     = src/main.cpp src/livro.cpp src/usuario.cpp src/emprestimo.cpp

all: $(TARGET)

$(TARGET): $(SRCS)
$(CXX) $(CXXFLAGS) -o $(TARGET) $(SRCS)

clean:
rm -f $(TARGET)
```

Para compilar e executar:

```
make          # compila
make clean    # remove o executável
./meu_sistema # executa
```

2.3 O que é compilação incremental (próximos trabalhos)

A versão acima recompila **todos** os arquivos a cada `make`. Em projetos grandes, isso é lento. A versão correta compila cada `.cpp` separadamente em um arquivo `.o` (objeto) e só recompila os que mudaram:

```
# Makefile – versão incremental (para referência; use a partir do TP2)

CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -g
TARGET   = meu_sistema
SRCS     = src/main.cpp src/livro.cpp src/usuario.cpp
OBJS     = $(SRCS:.cpp=.o)

all: $(TARGET)

$(TARGET): $(OBJS)
$(CXX) $(CXXFLAGS) -o $@ $^

%.o: %.cpp
$(CXX) $(CXXFLAGS) -c -o $@ $<

clean:
rm -f $(TARGET) $(OBJS)
```

A partir do TP2, esperamos a versão incremental.
Para o TP1, a versão simples da seção 2.2 é suficiente.

3. CMakeLists.txt

3.1 Como funciona

CMake é um **gerador de build**: ele lê um arquivo **CMakeLists.txt** e gera o Makefile (ou projeto do VS) automaticamente. Você nunca edita o Makefile gerado — só o CMakeLists.txt.

O fluxo tem dois passos:

```
# Passo 1: configurar (gera os arquivos de build na pasta "build/")
cmake -B build

# Passo 2: compilar
cmake --build build

# Executar
./build/meu_sistema
```

A pasta `build/` contém arquivos gerados automaticamente — adicione-a ao `.gitignore` e nunca a versione.

3.2 Exemplo mínimo

```
cmake_minimum_required(VERSION 3.20)
project(meu_sistema CXX)

# Padrão C++17 obrigatório
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Warnings e modo debug com sanitizers
if(CMAKE_BUILD_TYPE STREQUAL "Debug")
    add_compile_options(-Wall -Wextra -fsanitize=address,undefined -g)
    add_link_options(-fsanitize=address,undefined)
endif()

# Executável: liste TODOS os seus .cpp aqui
add_executable(meu_sistema
    src/main.cpp
    src/livro.cpp
    src/usuario.cpp
    src/emprestimo.cpp
)

# Diretório de headers (se usar subpasta include/)
target_include_directories(meu_sistema PRIVATE src)
```

Para compilar em modo Debug (com AddressSanitizer habilitado):

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/meu_sistema
```

3.3 Por que CMake é o padrão da disciplina

O CMake gera automaticamente o arquivo `compile_commands.json` quando você adiciona `-DCMAKE_EXPORT_COMPILE_COMMANDS=ON` na configuração. Esse arquivo é o que o VS Code usa para IntelliSense preciso, navegação de código e detecção de erros em tempo real — sem ele, o editor não sabe quais flags você está usando.

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
# Gera build/compile_commands.json - usado pelo clangd / IntelliSense
```

4. Qual usar no TP1?

	Makefile (simples)	CMakeLists.txt
Complexidade para escrever à mão	Baixa (10–15 linhas)	Média (15–20 linhas)
Funciona sem instalar nada extra	✓ make já vem com build-essential	✓ cmake já instalado no laboratório
IntelliSense no VS Code	✗ sem compile_commands.json automático	✓ com -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
AddressSanitizer em modo Debug	Manual (adicionar flags no CXXFLAGS)	Automático via if(Debug)
Padrão da disciplina (TP2 em diante)	✗	✓
Recomendado para o TP1	✓ aceitável	✓ preferencial

Recomendação: use CMake. O LLM gera o arquivo em segundos a partir do prompt da seção seguinte, e você ganha IntelliSense e sanitizers de graça.
Se tiver qualquer problema com CMake, use Makefile — ambos são aceitos no TP1.

5. Usando LLM para gerar os arquivos do seu projeto

Você não precisa memorizar a sintaxe do CMake ou do Makefile. Use LLM (Claude, ChatGPT, etc.) com os prompts abaixo — basta substituir as partes em **[COLCHETES]** pelo que é do seu projeto.

5.1 Prompt para CMakeLists.txt

```
Gere um CMakeLists.txt para um projeto C++17 com as seguintes características:
```

- Nome do projeto: [NOME DO SEU PROJETO]
- Executável de saída: [NOME DO EXECUTAVEL]
- Arquivos .cpp: [LISTE SEUS ARQUIVOS, ex: src/main.cpp src/livro.cpp]
- Headers na pasta: [src/ OU include/ - o que você usa]

Requisitos:

- cmake_minimum_required VERSION 3.20
- C++17 obrigatório
- Flags -Wall -Wextra em todos os builds
- AddressSanitizer e UndefinedBehaviorSanitizer habilitados em modo Debug
- Exportar compile_commands.json automaticamente

Gere apenas o CMakeLists.txt, sem explicações adicionais.

Depois de receber o arquivo, salve como `CMakeLists.txt` na raiz do projeto e teste:

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build
# Se compilar sem erros: está pronto.
```

5.2 Prompt para Makefile

```
Gere um Makefile para um projeto C++17 com as seguintes
```

características:

- Executável de saída: [NOME_DO_EXECUTAVEL]
- Arquivos .cpp: [LISTE SEUS ARQUIVOS, ex: src/main.cpp src/livro.cpp]
- Headers na pasta: [src/ OU include/]

Requisitos:

- Compilador: g++
- Flags: -std=c++17 -Wall -Wextra -g
- Alvo "all" compila tudo em um único comando (versão simples, sem .o)
- Alvo "clean" remove o executável
- Indentação com TAB real (não espaços)

Gere apenas o Makefile, sem explicações adicionais.

Depois de receber o arquivo, salve como Makefile (sem extensão) na raiz do projeto e teste:

```
make
# Se compilar sem erros: está pronto.
make clean # para limpar quando quiser
```

5.3 Adicionando um novo arquivo .cpp ao projeto

Cada vez que você criar uma nova classe (novo .cpp), precisa informar o sistema de build. Não precisa pedir ao LLM novamente — edite você mesmo:

CMakeLists.txt	Makefile
<pre># No CMakeLists.txt, adicione # o novo .cpp em add_executable: add_executable(meu_sistema src/main.cpp src/livro.cpp src/nova_classe.cpp # <-- novo) # Depois recompila: # cmake --build build</pre>	<pre># No Makefile, adicione # o novo .cpp em SRCS: SRCS = src/main.cpp \ src/livro.cpp \ src/nova_classe.cpp # Depois recompila: # make</pre>

5.4 Se a compilação falhar após gerar o arquivo

Copie a mensagem de erro do terminal e pergunte ao LLM:

```
O CMakeLists.txt que você gerou produz este erro ao compilar:

[COLE O ERRO AQUI]

Minha estrutura de arquivos é:
[COLE A SAÍDA DE: find . -name "*.cpp" -o -name "*.hpp" | sort]

Corrija apenas o CMakeLists.txt.
```

Lembre: siga o Guia de Uso Inteligente de IA já distribuído. **Você é responsável por entender o arquivo gerado.** Se o LLM gerar algo que você não entende, pergunte: "Explique linha a linha o que esse CMakeLists.txt faz."

6. .gitignore recomendado

Adicione este arquivo na raiz do repositório para não versionar arquivos gerados:

```
# Arquivos de build — nunca versionar
build/
cmake-build-*/

# Executáveis
meu_sistema
*.out
*.o

# VS Code (opcional — pode versionar se quiser compartilhar config)
.vscode/

# macOS
```

7. Resumo em 4 passos

1. Defina os arquivos `.cpp` do seu projeto (um por classe, mais `main.cpp`).
2. Use o prompt da Seção 5.1 (CMake) ou 5.2 (Makefile), preenchendo os colchetes com seus dados.
3. Salve o arquivo gerado na raiz do projeto, compile e confirme que funciona.
4. A cada nova classe adicionada, atualize a lista de `.cpp` no arquivo de build conforme a Seção 5.3.

Comando	O que faz
<code>cmake -B build -DCMAKE_BUILD_TYPE=Debug</code>	Configura o projeto CMake em modo Debug
<code>cmake --build build</code>	Compila o projeto
<code>./build/nome_do_executavel</code>	Executa o programa
<code>make</code>	Compila o projeto (Makefile)
<code>./nome_do_executavel</code>	Executa o programa (Makefile)
<code>make clean</code>	Remove o executável (Makefile)